

MSCR Tracker — Session Handoff Document

Intel RealSense D435i · Real-time 3-D Rod Segmentation & Tip Tracking

Project Goal

- Segment, localize, and track a black Magnetic Soft Continuum Robot (MSCR)
- in real-time using an Intel RealSense D435i depth camera.
- Output: 3-D arc length (mm) and tip (X, Y, Z) in the camera frame.

Repository Layout (/home/dozie/mscr_calibration/)

- mscr_tracker.py — main tracker (1 083 lines, all logic here)
- mscr_performance_report.py — standalone PDF analytics generator
- generate_handoff.py — this handoff document generator
- config.py — camera/board constants
- calibrate.py — intrinsic calibration pipeline
- capture.py — RealSense frame capture utility
- results/calibration.yaml — calibrated K matrix + distortion coeffs
- results/calibration_report.pdf — full calibration report

Hardware

- Camera : Intel RealSense D435i (USB 3, /dev/video*)
- Default stream : 848 × 480 @ 30 fps (colour BGR8 + depth Z16, aligned)
- Emitter : off by default (--power 0) — passive IR to avoid blooming
- Rod : black, slender, highly flexible cylindrical MSCR

Calibration (already done — do NOT re-run unless camera is moved)

- RMS reprojection error : 0.734 px (excellent; < 1.0 px is acceptable)
- Intrinsics (1280×720): fx=892.08, fy=893.93, cx=646.65, cy=367.62
- Distortion (k1..k3, p1, p2): [0.108, -0.463, 0.007, 0.006, 0.427]
- Depth scale : 0.001 m/unit (1 mm per raw depth unit)

– Intrinsic are fetched live from rs2 stream — calibration.yaml is reference only

Quick-start Commands

- python mscr_tracker.py --debug # live view, no report
- python mscr_tracker.py --debug --report # live view + PDF on exit
- python mscr_tracker.py --debug --report my.pdf --threshold 60
- python mscr_performance_report.py --frames 300 --out perf.pdf

Pipeline Architecture

process_frame(color_bgr, depth_filtered, depth_raw) → TrackResult

1

Raw depth snapshot

depth_raw = asarray(depth_frame) before any post-processing.
Zero-depth pixels (rod absorbs IR → no return) are captured here.
Used to build rod_holes mask in Step 2.

2

Depth filtering + gate mask

rs.spatial_filter + rs.temporal_filter applied to depth_frame.
depth_gate(): pixels confirmed outside [depth_min, depth_max] → excluded.
rod_holes (from raw, Step 1) force-OR'd into gate → rod never gated out.

3

Dark-pixel segmentation

BGR→Gray; THRESH_BINARY_INV at dark_threshold (default 55).
Otsu fallback on clipped histogram when too few dark pixels found.
mask = dark AND gate.

4

Morphological cleanup + CC selection

MORPH_OPEN (1 iter) → MORPH_CLOSE (2 iter) on 3×3 ellipse SE.
_largest_elongated_cc(): fitEllipse on each CC; prefer highest eccentricity
(threshold 0.80) and area in [min_area_px, 5% of frame].

5

Skeletonisation + branch pruning

skimage.morphology.skeletonize → 1-px-wide centerline.
_prune_skeleton_branches(): remove arms < 8 px reaching a junction (3 passes).
Result: clean unbranched curve.

6

Endpoint detection + ordered path

_find_endpoints(): degree-1 pixels; fallback to most-distant pair.
_assign_base_tip(): uses smooth_tip_px prior (EMA) to prevent base/tip swap;
~~falls back to entry-border heuristic (default: bottom=base).~~
_order_path(): greedy nearest-neighbour walk base→tip (no DFS backtracking).

7

3-D projection + spline

rs2.deproject_pixel_to_point() for each skeleton pixel using rs2 intrinsics.
np.interp fills zero-depth gaps per axis.
splprep (k=3) → arc length uniform resample to n_resample=200 pts.

8

Arc length + tip extraction

arc_mm = sum(||P_{i+1}-P_i||) × 1000. tip_xyz_mm = last point × 1000.
EMA (alpha=0.25) smooths tip and arc across frames.
Spike rejection: |tip_jump| > max_jump_mm=30 → hold previous estimate.

Key Classes & Public API

TrackerParams (dataclass)

- width / height / fps
 - 848×480 @ 30 — stream resolution
- laser_power = 0
 - emitter off; set >0 for active IR
- depth_min_m / depth_max_m
 - 0.05 – 0.80 m depth gate
- depth_dilation_px = 6
 - erode exclusion mask near rod edges
- dark_threshold = 55
 - inverse threshold (lower = darker only)
- min_eccentricity = 0.80
 - CC eccentricity gate; rods ≈ 0.90+
- min_skel_pts / max_skel_pts
 - 15 – 1000 valid skeleton pixels
- min_valid_depth_frac = 0.02
 - ≥2% of skel pts must have depth
- n_resample = 200
 - final spline sample count
- spline_smooth = 2.0
 - splprep $s = N \times \text{smooth_factor}$
- entry = 'bottom'
 - 'top'|'bottom'|'left'|'right'
- roi_pad_px = 60
 - padding added to temporal ROI
- max_lost_frames = 8
 - before ROI resets to full frame
- ema_alpha = 0.25
 - tip/arc smoothing rate
- max_jump_mm = 30.0
 - spike rejection threshold (mm)
- debug = False
 - enable side-by-side debug window

MSCRTracker (class)

- __init__(params)
- set_intrinsics(rs.intrinsics)
 - must call before process_frame
- process_frame(color, depth,
 - depth_raw=None) → TrackResult
- run(report_path=None) → generator
 - yields TrackResult per frame
 - writes PDF on exit if report_path set

TrackResult (dataclass)

- valid : bool
 - True = 3-D arc+tip computed
 - False + centerline_px non-empty → skeleton visible, no 3-D depth
 - False + centerline_px empty → lost
- centerline_px : (N,2) float32
 - (col, row) full-image coords
- centerline_3d : (N,3) float64
 - (X, Y, Z) in mm, camera frame
- arc_length_mm : float
 - EMA-smoothed 3-D arc length
- tip_px : (col, row)
 - 2-D tip position
- base_px : (col, row)
 - 2-D base position
- tip_xyz_mm : (X, Y, Z)
 - EMA-smoothed 3-D tip in mm
- mid_xyz_mm : (X, Y, Z)
 - 50% arc-length point in mm
- debug_frame : ndarray | None
 - side-by-side BGR image for imshow

Terminal Status Lines

- arc=... mm | tip=(X,Y,Z) mm
 - valid=True, full 3-D tracking
- [skeleton Npx — no 3D depth]
 - rod segmented; depth unavailable
- [no detection]
 - rod not found in frame

Coordinate Conventions

- 2-D : (col, row) = OpenCV (u, v)
- 3-D : RealSense camera frame
 - X = right, Y = down, Z = forward
- Units : metres internally, mm output

Bugs Fixed & Non-Obvious Design Decisions

Critical context — do not revert these without understanding the rationale

BUG (fixed) — spatial_filter erases the rod

- rs.spatial_filter fills zero-depth holes (where rod absorbs IR) with
 - interpolated background depth values. After filtering those pixels carry
 - background-range depths → depth gate excludes them → rod vanishes from mask.
- FIX: depth_raw captured BEFORE filters. rod_holes=(raw==0) is OR'd into
- the gate unconditionally so the rod silhouette always survives filtering.

BUG (fixed) — cv2.erode on empty ROI array crashes

- When temporal bbox drifts to frame edge the ROI slice becomes zero-size.
- FIX: _depth_gate() guards depth.size==0 early return; also checks ROI
- dimensions against structuring element size before calling erode.
- process_frame() resets bbox to full frame when rx2<=rx1 or ry2<=ry1.

BUG (fixed) — base/tip endpoint swap when the rod bends

- Anchoring on the MOVING tip flips base/tip when the rod bends sharply.
- FIX: anchor on the STATIC base instead. _anchor_base_px is a slow EMA
- (base_ema_alpha=0.10) of the fixed mount; _assign_base_tip() calls the
- endpoint nearest that anchor the base. The base barely moves, so it
- cannot flip. Entry-border heuristic is bootstrap-only; anchor clears on
- a full-frame reset so a repositioned fixture re-acquires cleanly.

BUG (fixed) — DFS ordering detours through skeleton side-branches

- skimage.skeletonize() produces short side-spurs at blob irregularities.
- DFS could backtrack and traverse a spur, producing a non-monotone path.
- FIX: _prune_skeleton_branches() removes arms <8px reaching a junction
- (3 passes). _order_path() replaced with greedy nearest-neighbour walk
- that cannot backtrack and always advances toward the target endpoint.

DESIGN — min_valid_depth_frac = 0.02 (intentionally very low)

- The black rod absorbs nearly all structured IR light. Even with emitter on,
- most skeleton pixels return depth=0. Linear interpolation from the tiny
- fraction with real returns gives usable 3-D shape; the spline smooths it.
- Raising this threshold above ~0.05 will cause constant 3-D detection failure.

DESIGN — EMA gate display (alpha=0.35) for right debug panel

- Raw depth gate re-rendered per frame produces per-pixel flicker even with
- temporal depth filtering. Gate image is blended across frames (35/65 mix)
- before applyColorMap. This is visualisation-only; the actual gate mask
- used for segmentation is computed fresh each frame (no smoothing there).

Robustness: Tip-flip, ROI Explosion & Hand Intrusion

Latest hardening pass — manual manipulation with hands in the frame

Tip / base flip → static base anchor

- `_anchor_base_px` : slow EMA (`base_ema_alpha=0.10`) of the fixed mount.
- `_assign_base_tip()` classifies the endpoint nearest the anchor as base.
- `base_lock_dist_px=120` : detection rejected if nearest endpoint is farther.

ROI explosion → plausibility guard + incremental widening

- `_roi_plausible()` compares bbox DIAGONAL span (not area — a thin rod's
- area collapses to ~ 0 when vertical/horizontal, giving false explosions).
- Rejects bbox if span ratio $> \text{roi_max_growth}$ (3.0) or centre shift $>$
- `roi\_max\_shift_px` (140). On reject → `_soft_fail()` HOLDS the last good
- curve (no spike, no hard LOST). When lost, the ROI grows by
- `roi\_expand_px` (30) per frame instead of snapping to full frame.

Hand / arm intrusion → slender isolation + base walk

- `_isolate_slender()`: morphological OPEN with a kernel of diameter
- `rod\_max\_width_px+3` (~ 15 px) — between rod width (≤ 12) and finger width
- (≥ 18). OPEN keeps blobs WIDER than the kernel (the hand); subtracting
- that from the mask leaves only the slender rod. CRITICAL: the kernel
- must be SMALLER than a finger or fingers leak back into the thin mask.
- `_largest_elongated_cc(base_anchor_roi)`: among elongated candidates,
- picks the component NEAREST the base anchor — walks down the rod from
- its fixed base, ignoring any attached hand component.
- `hand\_thick\_frac=0.30` sets `self._hand_present` when $> 30\%$ of fg is thick.

Hand-present adaptive behaviour (`self._hand_present`)

- ROI guards relax $\times \text{hand_relax_factor}$ (2.5) — manual motion is larger.
- Depth-proxy search widens to `depth_search_r_hand` (14) — a hand perturbs
- depth near the rod, so a bigger neighbourhood is needed for valid depth.
- `depth_dilation_px` lowered $6 \rightarrow 3$ so the gate erosion doesn't eat thin rod
- data at boundaries when a hand is adjacent.

Expected behaviour under occlusion

- When a finger crosses the rod, the visible rod is segmented up to the
- occlusion; the 3-D spike guard HOLDS the last tip (`valid=False`, curve
- still drawn) rather than jumping. Full LOST only if the base region
- itself is fully covered — recovers within `max_lost_frames` (8).

Tuning Guide — What to Change When

Symptom	Parameter / fix
Rod not detected at all	Lower dark_threshold (try 65-80); check depth_min_m not too high
Hand grabs the rod / tracking lost on touch	slender_isolation on; tune rod_max_width_px (< finger width); base anchor walks rod
Fingers leak into mask	rod_max_width_px too large — lower it so kernel stays below finger width
Centerline jumps/flips	EMA already active; check entry= setting; inspect_smooth_tip_px reset
Skeleton too short	Lower min_skel_pts; increase morph_close_iter
[skeleton — no 3D depth] constant	Lower min_valid_depth_frac (<0.02); try --power 150 (enable emitter)
3-D arc length noisy	Lower ema_alpha (more smoothing); raise n_resample; raise spline_smooth
Tip jumps large	Lower max_jump_mm; lower ema_alpha
Slow / low FPS	Switch to 848×480 (default); raise min_skel_pts to skip small blobs
Rod exits frame and re-enters	max_lost_frames=8 resets ROI; roi_pad_px=60 gives search margin
Rod near frame edge gets clipped	Raise roi_pad_px; check depth_dilation_px not too large

Recommended Workflow for New Scene

- 1. Run with --debug and observe right panel
 - Is the rod region black (zero depth) or coloured?*
- 2. If rod not segmenting: lower dark_threshold
 - Try 60, 65, 70 in steps*
- 3. If false positives: raise min_eccentricity
 - Inspect CC eccentricity with print statements*
- 4. If 3-D always fails: lower min_valid_depth_frac
 - 0.01 is safe lower bound given spline interpolation*
- 5. If tip unstable: tune ema_alpha
 - 0.15 = heavy smoothing, 0.40 = fast response*

Environment Variables / Paths

- Working dir : /home/dozie/mscr_calibration/
- Python env : system Python3, no venv detected
- Key deps : pyrealsense2, opencv-python,
 - scipy, scikit-image, numpy, matplotlib*
- Camera rules : 99-realsense-libusb.rules
 - (copy to /etc/udev/rules.d/ if device not seen)*

Known Limitations / Open Tasks

- No extrinsic calibration (camera→robot base)
 - tip_xyz_mm is in camera frame, not robot frame*
- No handling of rod occlusion by hand/fixture
- Entry border heuristic fails for horizontal rod
 - Use external base-point prior if available*
- Emitter-off mode gives sparse depth on rod
 - Emitter-on may help but increases IR noise*
- Performance report requires matplotlib + camera
 - Cannot be generated from saved video yet*

Next Steps & Integration Notes

Immediate: verify 3-D accuracy

- Hold the rod at a known distance (e.g. 300 mm from camera lens).
- Check `tip_xyz_mm.Z` $\approx 300 \pm 5$ mm. If not, verify `depth_scale` from `calibration.yaml`.
- Measure known arc length manually and compare to `arc_length_mm` output.

Extrinsic Calibration (camera→robot-base transform)

- Needed to express `tip_xyz_mm` in the robot workspace frame.
- Approach: fix a calibration target at a known robot position, observe it
- in the camera frame, solve for `R`, `t` using `cv2.solvePnP` or hand-eye calibration.
- Store result in `results/extrinsic.yaml` alongside `calibration.yaml`.

Control Integration

- `TrackResult.tip_xyz_mm` provides (X, Y, Z) in mm at 30 fps.
- Consume the `run()` generator in a control loop:
 - *for result in tracker.run():*
 - *if result.valid:*
 - *send_to_controller(result.tip_xyz_mm, result.arc_length_mm)*
- EMA smoothing (`ema_alpha=0.25`) already applied — additional filtering optional.
- `result.centerline_3d` is (200, 3) float64 — full shape curve for shape control.

Generated by `generate_handoff.py` · `/home/dozie/mscr_calibration/`

Performance Benchmarking

- `python mscr_tracker.py --debug --report my_session.pdf`
- Report auto-generated on Q/ESC exit. Contains:
 - *Detection rate, latency, arc length distribution, tip jitter (5 pages)*
- Standalone: `python mscr_performance_report.py --frames 300 --out perf.pdf`

Regenerating This Document

- `python generate_handoff.py`
- Output: `mscr_handoff.pdf` in `/home/dozie/mscr_calibration/`
- Edit `generate_handoff.py` to add new findings before running.